

AI-Assisted Software Development

Module 8 — Business AI in Practice · Notebook 30

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

Contents

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

The modern IDE landscape (2026)

Tool	AI assistance	Best for
VS Code + Copilot	inline completion + chat	broad default; great Jupyter support
Cursor	chat-first; multi-file refactors	rewriting larger chunks; agentic edits
PyCharm + JetBrains AI	completion + assistant panel	large, refactor-heavy projects

Recommendation for beginners

VS Code + Copilot or **Cursor** — the most forgiving combinations to learn the workflow on.

Three ways the assistant helps

- **Inline completion** — grey ghost-text as you type. Fast, low-effort; good for small, repetitive code.
- **Chat** — a side panel where you request a multi-line / multi-file change. Slower, higher-effort; good for new functions and refactors.
- **Agent mode** — the assistant edits files and runs commands on its own. Most powerful; needs the most review (see Module 9).

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

Git — nine commands cover 90 % of daily work

Command	What it does
<code>status / diff</code>	see what changed
<code>add / commit</code>	stage and snapshot
<code>pull / push</code>	sync with the remote
<code>checkout -b / checkout</code>	make / switch a branch
<code>merge</code>	combine a branch back in

Common beginner mistake

Force-pushing (`git push --force`) over the main branch destroys history other people have built on. Force-push only your *own* feature branches.

Pull requests — the real unit of professional work

A good PR has

- A **short title** that reads like a sentence: “Add retry-with-backoff to the LLM client.”
- A **2-paragraph description**: what changed, why, and what testing was done.
- **Small scope**. A 30-line PR is reviewed in 5 minutes; a 3,000-line PR sits for a week.
- **Tests**, or at least a screenshot of having run the new code.

AI-generated PRs are no exception — arguably *more* important to keep small, because the reviewer’s job is to check what the AI couldn’t.

Outline

- 1 IDEs
- 2 Git
- 3 Prompting**
- 4 Review
- 5 When not

Five prompt patterns for code

- ➊ **Give it what it can't see** — paste the surrounding code, the schema, the error.
- ➋ **Ask for one thing**, explicitly — one function or change per turn, then iterate.
- ➌ **Be specific about constraints** — Python version, libraries to use / avoid, style, performance, error semantics.
- ➍ **Ask for tests** — “add a pytest that covers the empty-input case.”
- ➎ **Ask for alternatives when designing** — “give me three storage options with trade-offs.”

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review**
- 5 When not

Four failure modes of AI-generated code

- **A — Hallucinated APIs.** Calls a function or argument that does not exist.
- **B — Plausible but wrong.** Looks right; the logic is subtly off (a “weighted average” that isn’t).
- **C — Architectural drift.** Ignores the codebase’s patterns — `print` where everything else uses `logging`.
- **D — Silent over-confidence.** “Safely” swallows errors with a bare `except` and hides the failure.

The 60-second review checklist

Run this on every AI-generated change before you accept or merge

- ❶ **Does it run?** Even just the import line.
- ❷ **Are there tests?** If not, ask for them.
- ❸ **Does it use our libraries / patterns?** Grep for one similar function.
- ❹ **Does it swallow errors?** Search the diff for `except Exception`, or `{}`, `pass`.
- ❺ **Does it do exactly what was asked** — no more, no less?

60 seconds; catches roughly 80 % of the failures. The remaining 20 % is what real testing and code review are for.

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

When *not* to use the AI assistant

Mental model

AI assistants are most valuable when you *could* write the code yourself but would rather not. They are least valuable when you don't yet understand the problem.

- Skip it when the task is to *learn* the concept — let it explain, but write the first version yourself.
- Skip it for security-critical or correctness-critical logic you can't fully review.
- **Companion notebook:** hands-on Git, PR, and review exercises are in **Notebook 30**; the course recommends it *early* so the workflow pays off everywhere.